Statistical Machine Learning - HW 1 (Solutions)

Problem 1

Let $\mathbf{X} = (X_1, X_2, X_3)^\top \sim MVN(\mu, \Sigma)$ with

$$\mu = \begin{pmatrix} 1 \\ -1 \\ -2 \end{pmatrix}, \qquad \Sigma = \begin{pmatrix} 4 & 0 & -1 \\ 0 & 5 & 0 \\ -1 & 0 & 2 \end{pmatrix}.$$

Fact (MVN): If $(X_1, \dots, X_p)$ is multivariate normal, then **zero covariance implies independence** for any pair of components: $X_i \perp X_j \iff \mathrm{Cov}(X_i, X_j) = \Sigma_{ij} = 0$. More generally, two subvectors are independent iff their *cross-covariance block* is the zero matrix.

(a) X_1 **and** X_2

$\mathrm{Cov}(X_1, X_2) = \Sigma_{12} = 0; \Rightarrow; X_1 \perp X_2.$

(b) X_1 **and** X_3

$\mathrm{Cov}(X_1, X_3) = \Sigma_{13} = -1 \neq 0; \Rightarrow; X_1 \not\perp X_3.$

(c) X_2 **and** X_3

$\mathrm{Cov}(X_2, X_3) = \Sigma_{23} = 0; \Rightarrow; X_2 \perp X_3.$

(d) (X_1, X_3) and X_2

The cross-covariances are $\mathrm{Cov}(X_1, X_2) = \Sigma_{12} = 0, \quad \mathrm{Cov}(X_3, X_2) = \Sigma_{32} = 0$. So the cross-covariance block between (X_1, X_3) and X_2 is $(0, 0)^\top$, hence $(X_1, X_3) \perp X_2$.

(e) X_1 and $X_1 + 3X_2 - 2X_3$

For multivariate normal variables, two linear combinations are independent iff their covariance is 0. Let $Y = X_1 + 3X_2 - 2X_3$. Then $\mathrm{Cov}(X_1, Y) = \mathrm{Cov}(X_1, X_1) + 3, \mathrm{Cov}(X_1, X_2) - 2, \mathrm{Cov}(X_1, X_3)$. Using $\mathrm{Var}(X_1) = \Sigma_{11} = 4$, $\Sigma_{12} = 0$, and $\Sigma_{13} = -1$: $\mathrm{Cov}(X_1, Y) = 4 + 3(0) - 2(-1) = 6 \neq 0$. Therefore, $X_1 \not\perp (X_1 + 3X_2 - 2X_3)$.

Problem 2

Suppose $\mathbf{X}$ is a random vector with mean $\mu = \mathbb{E}[\mathbf{X}]$. Prove: $\mathbb{E}\big[(\mathbf{X}-\mu)(\mathbf{X}-\mu)^\top\big] = \mathbb{E}(\mathbf{X}\mathbf{X}^\top) - \mu\mu^\top$.

Proof. Expand: $(\mathbf{X} - \mu)(\mathbf{X} - \mu)^\top = \mathbf{X}\mathbf{X}^\top - \mathbf{X}\mu^\top - \mu\mathbf{X}^\top + \mu\mu^\top$. Take expectation and use $\mathbb{E}[\mathbf{X}] = \mu$. Therefore $\mathbb{E}(\mathbf{X}-\mu)(\mathbf{X}-\mu)^\top = \mathbb{E}\mathbf{X}\mathbf{X}^\top - \mathbb{E}\mathbf{X}\mu^\top - \mu\mathbb{E}\mathbf{X}^\top + \mu\mu^\top = \mathbb{E}\mathbf{X}\mathbf{X}^\top - \mu\mu^\top$.□

Problem 3

Let $\mathbf{Z} \sim MVN(\mathbf{0}, I_p)$, and let Σ be positive definite.

A general strategy is: find a matrix A such that $AA^\top = \Sigma$, then set $\mathbf{X} = \mu + A\mathbf{Z}$. Since $\mathbb{E}[\mathbf{Z}] = 0$ and $\mathrm{Cov}(\mathbf{Z}) = I_p$, $\mathbb{E}\mathbf{X} = \mu, \quad \mathrm{Cov}(\mathbf{X}) = A, I_p, A^\top = AA^\top = \Sigma$.

(a) Spectral decomposition

If $\Sigma = V\Lambda V^\top$ with $V^\top V = I_p$ and diagonal $\Lambda = \mathrm{diag}(\lambda_1, \ldots, \lambda_p)$, define $\Lambda^{1/2} = \mathrm{diag}(\sqrt{\lambda_1}, \ldots, \sqrt{\lambda_p}), \quad A = V\Lambda^{1/2}$. Then $AA^\top = V\Lambda^{1/2}\left(V\Lambda^{1/2}\right)^\top = V\Lambda^{1/2}\Lambda^{1/2}V^\top = V\Lambda V^\top = \Sigma$.

(b) Cholesky decomposition

If $\Sigma = LL^\top$ where L is lower-triangular with positive diagonal, take $A = L$. Then trivially $AA^\top = LL^\top = \Sigma$.

Problem 4

Generate 1000 samples using both methods from Problem 3, and compare sample covariance matrices.

Python code:

```python
import numpy as np

mu = np.array([1.0, -1.0, -2.0])
Sigma = np.array([
    [4.0, 0.0, -1.0],
    [0.0, 5.0,  0.0],
    [-1.0, 0.0, 2.0]
])

n = 1000
p = 3
rng = np.random.default_rng(123)

# ---- Method (a): spectral decomposition
eigvals, V = np.linalg.eigh(Sigma)          # Sigma = V diag(eigvals) V^T
A_spec = V @ np.diag(np.sqrt(eigvals))      # A_spec A_spec^T = Sigma
Z = rng.standard_normal((n, p))
X_spec = mu + Z @ A_spec.T

S_spec = np.cov(X_spec, rowvar=False, bias=False)

# ---- Method (b): Cholesky decomposition
L = np.linalg.cholesky(Sigma)               # Sigma = L L^T
Z2 = rng.standard_normal((n, p))
X_chol = mu + Z2 @ L.T

S_chol = np.cov(X_chol, rowvar=False, bias=False)
```

```
# Compare
print("Target Sigma:\n", Sigma, "\n")
print("Sample cov (spectral):\n", np.round(S_spec, 3), "\n")
print("Sample cov (cholesky):\n", np.round(S_chol, 3), "\n")

print("Frobenius error (spectral):", np.linalg.norm(S_spec - Sigma, ord="fro"))
print("Frobenius error (cholesky):", np.linalg.norm(S_chol - Sigma, ord="fro"))
```

R Code:

```
    n <- 1000; mu <- c(1,-1,-2);
    Sig <- matrix(c(4,0,-1,0,5,0,-1,0,2), nr=3)
    Z <- matrix(rnorm(n*length(mu)),nr=3)
    X1 <- t(mu + eigen(Sig)$vectors%*%diag(sqrt(eigen(Sig)$values))%*%Z)
    X2 <- t(mu + t(chol(Sig))%*%Z)
    cov(X1)
    cov(X2)
```

You should see both sample covariance matrices close to Σ, with small differences due to Monte Carlo variation.

Problem 5 (ISLP Chapter 2, Exercise 7)

We predict Y at the test point $x_0 = (X_1, X_2, X_3) = (0, 0, 0)$ using K-nearest neighbors (KNN) with Euclidean distance.

The training set (six observations) is:

Obs	X_1	X_2	X_3	Y
1	0	3	0	Red
2	2	0	0	Red
3	0	1	3	Red
4	0	1	2	Green
5	-1	0	1	Green
6	1	1	1	Red

(a) Distances to $x_0 = (0, 0, 0)$

For observation i with predictors $x_i = (x_{i1}, x_{i2}, x_{i3})$, the Euclidean distance is $d(x_i, x_0) = \sqrt{x_{i1}^2 + x_{i2}^2 + x_{i3}^2}$.

Compute each:

- Obs 1: $\sqrt{0^2 + 3^2 + 0^2} = 3$
- Obs 2: $\sqrt{2^2 + 0^2 + 0^2} = 2$
- Obs 3: $\sqrt{0^2 + 1^2 + 3^2} = \sqrt{10} \approx 3.162$
- Obs 4: $\sqrt{0^2 + 1^2 + 2^2} = \sqrt{5} \approx 2.236$
- Obs 5: $\sqrt{(-1)^2 + 0^2 + 1^2} = \sqrt{2} \approx 1.414$
- Obs 6: $\sqrt{1^2 + 1^2 + 1^2} = \sqrt{3} \approx 1.732$

Sorted by distance (smallest first):

Rank	Obs	Distance	Class
1	5	$\sqrt{2} \approx 1.414$	Green
2	6	$\sqrt{3} \approx 1.732$	Red
3	2	2	Red
4	4	$\sqrt{5} \approx 2.236$	Green
5	1	3	Red
6	3	$\sqrt{10} \approx 3.162$	Red

(b) Prediction with $K = 1$

The nearest neighbor is Obs 5 (Green), so the prediction is:

$\hat{Y}_{K=1} = \text{Green}$.

(c) Prediction with $K = 3$

The three nearest neighbors are Obs 5 (Green), Obs 6 (Red), Obs 2 (Red).
Majority vote is Red (2 out of 3), so:

$\hat{Y}_{K=3} = \text{Red}$.

(d) If the Bayes decision boundary is highly non-linear, should best K be large or small?

We would generally expect the best K to be **small**, because a small K yields a more flexible decision boundary (lower bias) that can adapt to strong non-linearities. A large K averages over many points and produces an overly smooth boundary (higher bias).

Python code